

8. 性能测试

性能是持续优化的质量目标!

目 录

CONTENTS

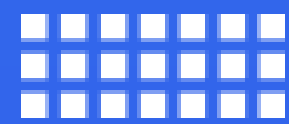
1

性能测试初步

2

性能测试进阶

8.1 性能测试初步



01

性能测试简介



性能测试简介

什么是性能测试？

性能测试是一种评估软件或系统在实际运行环境中表现的方法，通过模拟真实用户行为，评估系统在响应时间、吞吐量和资源利用率等指标的表现，确保其满足预定的性能需求。利用负载模拟和压力测试，分析系统在不同负载下的表现，发现并解决性能瓶颈，优化资源使用效率，保障系统高效运行。

思考：

- 性能测试在软件生命周期各阶段的目标与重点是什么？

性能测试类型与系统影响



软件生命周期中的性能测试

性能测试在软件生命周期各阶段的目标、重点和复杂性

- 需求分析阶段：目标是明确性能需求（如响应时间、并发用户数、吞吐量），重点在于收集和文档化性能指标，包括用户场景和业务流程。
- 设计阶段：评估系统架构的性能，设计非功能需求（如缓存策略、数据库优化），确保架构满足性能需求并识别潜在风险。
- 开发阶段：实施性能测试以发现和修复早期问题，重点在于单元性能测试和代码级优化。
- 测试阶段：
 - 单元测试：验证单个函数或方法的性能。
 - 集成测试：识别接口和集成点的性能瓶颈，验证组件交互性能。
 - 系统测试：全面评估系统性能，包括并发测试、负载测试和压力测试，确保端到端性能符合需求。

系统级性能测试

常见的性能测试类型

- 并发测试：模拟多用户同时操作，评估系统在高并发下的稳定性和响应能力，旨在发现并发引起的问题。
- 负载测试：在正常负载下逐步施压，评估系统性能极限，帮助了解系统容量并支持性能调优。
- 压力测试：模拟超预期负载，测试系统在极端条件下的崩溃点和恢复能力，找出最大承载能力。

这三种测试虽同属性能测试范畴，但目的和侧重点不同：并发测试关注多请求处理能力，负载测试评估长时间高负载下的稳定性，压力测试测试系统极限性能。它们共同构成全面评估系统性能的必要手段，确保系统在各种负载条件下的稳定性和响应速度，提升用户体验。

特殊性能场景

性能测试类型叠或同时进行

- 并发测试与负载测试结合：例如，电商网站在促销期间需同时处理大量用户请求（并发测试），并在长时间内保持高负载下的稳定性（负载测试）。
- 负载测试与压力测试结合：例如，社交平台在新版本发布时需应对突增的用户访问，逐步增加负载直至系统崩溃，以找出最大承载能力（压力测试）。
- 并发测试与压力测试结合：例如，流媒体平台在发布热门剧集时需处理大量用户同时播放视频（并发测试），并在短时间内承受极端压力（压力测试）。

性能测试执行阶段流程

执行阶段流程顺序

1. 确定测试环境：选项包括低规格服务器的生产系统子集、相同规格的较少服务器子集或完全复制的生产系统。需尽量保证测试环境与生产环境的一致性，但实际中因成本限制，通常难以搭建完全一致的测试环境，因此需通过策略解决。
2. 确定性能指标：包括响应时间、吞吐量、约束等，并明确成功标准。
3. 选择测试工具：根据需求选择合适的工具并执行测试。
4. 计划 and 设计测试：确定测试场景，考虑用户可变性、测试数据和目标指标，创建测试模型。

性能测试模型构建

性能测试分为两部分：

性能测试过程：执行测试并收集结果，形成测试文档。

模型构建过程：根据测试需求和生产系统信息构建性能模型，指导测试和结果生成。

具体模型：

业务模型：分析业务性能指标（如响应时间、吞吐量），确定测试范围、用户量、期望指标等。

数据模型：分析业务数据，评估数据规模、真实性，确定参数化策略和数据构造方法。

测试模型：包括测试计划、设计、执行、结束与汇总，以及技术准备和工具选型。

监控模型：确定监控指标（如CPU、内存、数据库、网络等），制定监控方案。

执行模型：制定测试环境部署方案，包括应用、负载和监控环境。

风险模型：识别测试过程中可能遇到的风险（如过程、人员、技术、监控风险）。

性能测试结果分析

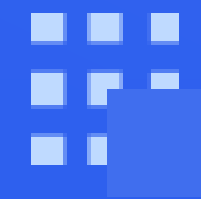
性能测试结果分析步骤

1. 检查测试环境和场景设置：确保环境正常、场景设置合理，避免因设置错误导致测试异常。
2. 分析测试结果：重点关注系统表现不正常的测试场景，深入分析暴露的问题。对压力下系统运行正常的结果无需过多分析，因其无法反映性能问题。
3. 调优与改进：通过测试报告进行性能结果分析，帮助研发、测试或运维人员快速定位瓶颈并进行调优。



02

性能测试工具



性能测试工具概述

性能测试工具的作用

工具在软件性能测试中具有重要作用，能够显著提升测试效率和准确性。通过自动化测试过程，这些工具减少了人为错误，同时节省了时间和资源。常见的性能测试工具包括负载测试工具、压力测试工具和监控工具等，它们能够模拟真实用户行为，检测系统在高负载下的性能瓶颈，并识别潜在问题。此外，性能测试工具提供详细的报告和分析功能，帮助开发团队深入理解系统性能表现，并根据测试结果进行优化和改进。因此，性能测试工具是确保软件系统高效、稳定运行的关键。



JMeter工具介绍

JMeter 概述与核心价值

JMeter (Apache JMeter) 是一款开源的性能测试工具，用于对多种协议和技术的应用程序进行负载测试和性能测量，帮助评估系统在不同负载下的表现。

核心优势：

- 多类型支持：测试 Web 应用、Web 服务、数据库等。
- 跨平台：基于 Java，支持多平台运行。
- 开源灵活：免费使用，支持定制和二次开发。
- 扩展性：通过 100+ 插件扩展功能，支持更多用例、增强核心功能并生成高级报告。

应用场景：

用于性能、负载和压力测试，帮助识别性能瓶颈、评估系统稳定性，为优化提供数据支持。

JMeter工具的使用

JMeter 的常用方法和步骤

1. 创建测试计划：定义线程组、采样器、监听器等。
2. 配置线程组：设置模拟用户数量、启动时间、循环次数等。
3. 添加采样器：定义请求类型，如 HTTP、FTP、JDBC 等。
4. 设置定时器和控制器：定时器控制请求间隔，控制器定义执行逻辑。
5. 使用监听器：收集并展示测试结果，如响应时间、吞吐量、错误率。
6. 执行测试并分析：执行测试计划，通过监听器查看结果，进行性能分析和优化。

常见问题与解决方案

使用JMeter进行性能测试时可能遇到的问题及解决方案

1. 内存不足：增加堆内存：修改 jmeter.bat (Windows) 或 jmeter (Unix/Linux) 中的 HEAP 参数。
2. 连接拒绝：检查服务器配置，确认端口和并发连接能力。
3. 响应时间过长：优化服务器性能，检查网络，调整线程数和循环次数。
4. GUI 冻结或无响应：避免 GUI 模式运行大量线程，改用命令行模式。
5. 无法找到元素（如 XPath/CSS）：检查页面源码，更新选择器匹配当前结构。
6. 脚本无法保存：分割脚本或检查文件系统权限。
7. 测试结果不精确：在非 GUI 模式下运行测试，确保测试环境一致。

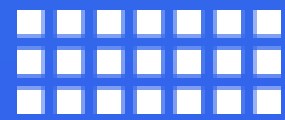
查找问题与解决方法

- 查看 JMeter 日志文件。
- 社区支持、官方文档、在线论坛（如 Stack Overflow）。

JMeter 性能测试最佳实践

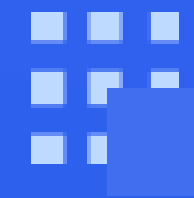
1. 模式选择: GUI 模式: 用于脚本创建和调试。非 GUI 模式: 执行测试, 减少资源消耗。
2. 配置优化: 合理设置线程数和循环次数, 避免负载过大。使用逻辑控制器、配置元件和定时器管理脚本流程。
3. 数据管理: 参数化模拟用户行为, 使用 CSV 文件提供测试数据。
4. 日志与监控: 启用日志记录, 使用监听器 (如聚合报告) 分析结果, 移除不必要的监听器。
5. 分布式测试: 模拟大量用户时, 使用分布式测试, 确保服务器时间同步。
6. 报告与集成: 生成详细报告分析性能瓶颈, 集成到持续流程中实现自动化测试。
7. 安全性: 避免脚本包含敏感信息, 使用安全连接保护数据。

8.2 性能测试进阶



01

多样性性能测试



多样性性能测试概述

什么是多样性性能测试？

多样性性能测试是指通过模拟多种不同的情境和条件，全面评估系统在各种环境下的性能表现的一种测试方法。它的核心在于多样化的测试场景和条件，以确保系统能够在不同的压力、配置和环境中稳定运行，并满足性能需求。

多样性性能测试情境和条件：

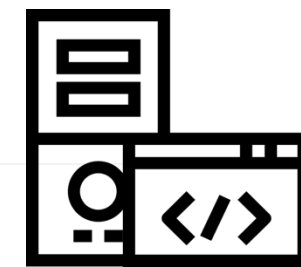
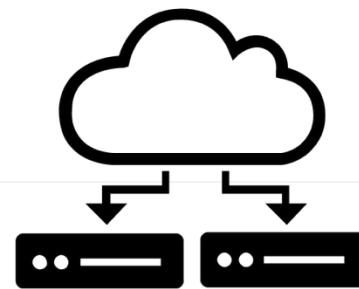
- 不同的负载：测试系统在不同负载下的表现，如响应时间、吞吐量和并发用户数等性能指标。
- 不同的网络条件：测试系统在网络延迟、带宽限制等不同网络条件下的表现。
- 不同的硬件和软件配置：测试系统在不同硬件和软件环境下的性能。

通过这种多样化的测试，测试人员可以更全面地了解系统的性能瓶颈及其原因，从而进行针对性的优化和改进。这种方法有助于确保系统在各种使用条件下的稳定性和可靠性。

性能测试中进行等价类场景划分

性能测试中进行等价类场景划分的作用

通过进行等价类场景划分法，可以有效减少测试数量，同时覆盖广泛的测试场景，确保系统在不同环境下的稳健性和可靠性。



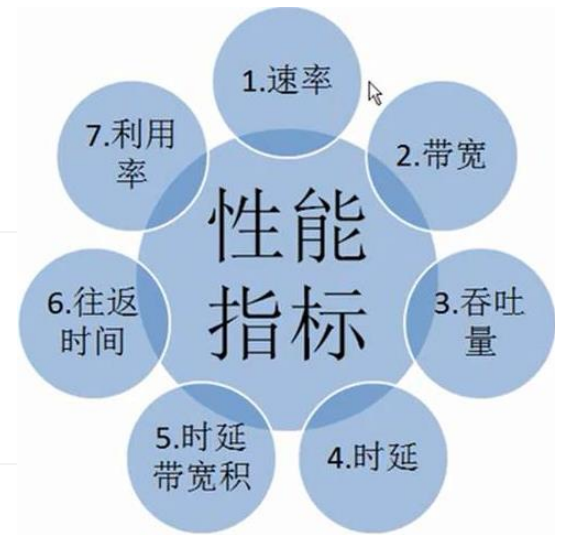
性能测试中的多样性策略应用

- 负载多样性：测试系统在不同负载情况下的表现，如正常负载、峰值负载和过载，以识别和解决性能瓶颈。
- 网络多样性：测试系统在不同网络条件下的表现，如网络延迟和带宽限制，以评估系统在不利网络环境下的性能。
- 硬件和软件多样性：测试系统在不同硬件和软件配置下的表现，包括不同的操作系统、数据库和服务器配置，以了解系统在不同配置下的性能表现和限制。

性能测试指标

性能指标的重要性

性能测试指标帮助明确性能测试的目标和期望结果，确保测试活动具有针对性和可衡量性。



性能指标的作用

- 指导测试设计：这些指标指导测试设计，包括测试场景、测试用例和测试数据的选择，确保测试覆盖关键性能方面并验证性能目标。
- 监控系统表现：在性能测试执行过程中，指标用于实时监控系统的性能表现，及时发现性能问题，调整测试策略，确保测试的有效性。
- 评估测试结果：性能测试指标是评估测试结果的基础。通过对指标数据的分析，可以确定系统是否满足性能要求，识别性能瓶颈，并为性能优化提供依据。

性能测试指标

性能指标的重要性

进行服务端性能测试时，需要通过性能测试的各项指标综合评价被测应用的性能。实际性能测试活动中，主要关注两方面的性能指标：业务指标和资源指标。

表 8.1: 性能测试的业务指标

业务指标	描述
并发用户数	某一物理时刻同时向系统提交请求的用户数。
平均响应时间	系统处理事务的响应时间的平均值，事务的响应时间是从客户端提交访问请求到客户端接收到服务器响应所消耗的时间，对于系统快速响应类页面，响应时间一般有最大响应时间，最小响应时间和平均响应时间。
事务成功	事务用于度量一个或者多个业务流程的性能指标，如用户登录、保存订单、提交订单操作均可定义为事务，单位时间内系统可以成功完成多少个定义的事务，在一定程度上反应了系统的处理能力，一般以事务成功率来度量。
超时错误率	主要指事务由于超时或系统内部其它错误导致失败占总事务的比率。
吞吐量	传输的数据量总和，也可以这样说在单次业务中，客户端与服务器端进行的数据交互总量。
吞吐率	吞吐量/传输时间，即单位时间内传输的数据量，也可以指单位时间内处理客户请求数量，它是衡量网络性能的重要指标。通常情况下，吞吐率用“字节数/秒”来衡量，也可以用“请求数/秒”和“页面数/秒”来衡量。
TPS	(Transaction Per Second) 每秒事务数，指服务器在单位时间内（秒）可以处理的事务数量，一般以“请求数/秒”为单位。
QPS	(Query Per Second) 每秒查询率，指服务器在单位时间内（秒）处理的查询请求速率。
PV	(Page View): 页面浏览量，通常是衡量一个页面甚至网站流量的重要指标。

表 8.2: 性能测试的资源指标

资源指标	描述
CPU 使用率	指用户进程与系统进程消耗的 CPU 时间百分比。
内存利用率	内存利用率 = (1-空闲内存/总内存大小) *100%。
磁盘 I/O	使用 %Disk Time（磁盘用于读写操作所占用的时间百分比）度量磁盘读写性能。
网络带宽	使用计数器“总字节数/秒”来度量，表示为发送和接收字节的速率。

性能需求

性能需求的定义

被测对象的性能需求通常在性能测试需求规格说明书中明确给出，以量化形式呈现，例如：

- 单位时间内的访问量
- 业务响应时间不超过某值
- 业务成功率不低于某值
- 硬件资源耗用需在合理范围内

性能测试结束后，测试人员需收集测试过程中的错误提示信息 and 测试结果监控指标数据，用于分析被测应用的性能表现。

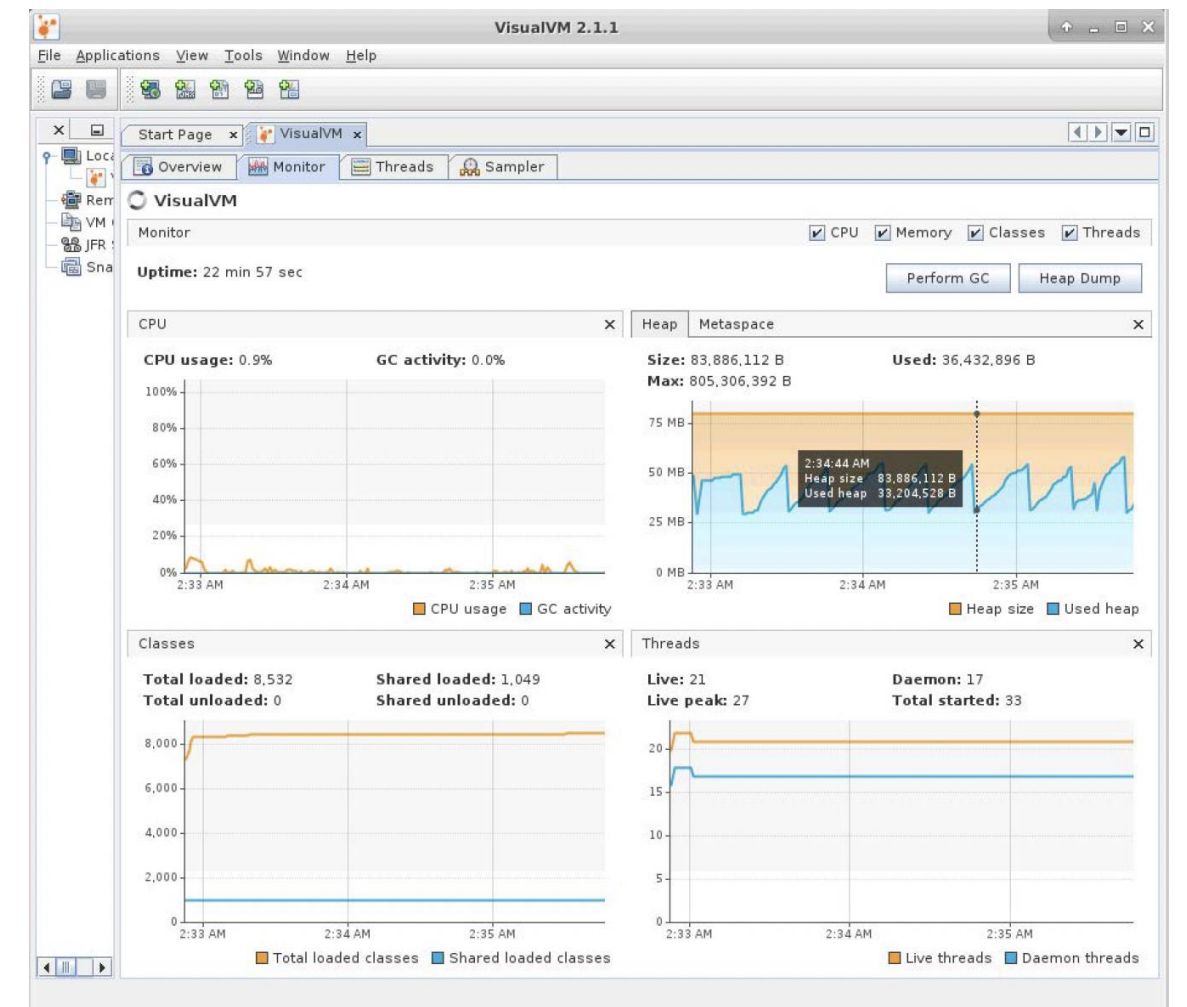
通过分析业务操作响应时间和内存资源使用情况来识别系统性能瓶颈

响应时间分析

- 使用平均事务响应时间图和事务性能摘要图，识别响应时间过长的的事务。
- 细分事务，分析页面组件性能。
- 若服务器耗时过长，通过服务器图查明原因；若网络耗时过长，通过“网络监视器”图定位问题。

内存分析

- 监控内存页交换速率：偶尔升高表示线程竞争内存；持续升高可能是内存瓶颈或命中率低。
- 在 Windows 中，若进程/私用位元组计数器和进程/工作单元计数器持续升高，且内存可用计数器持续降低，可能存在内存泄漏。
- 内存瓶颈的征兆包括：高换页率、进程不活动、磁盘活动频繁、CPU 利用率高、内存不足。



性能瓶颈查找

性能瓶颈查找顺序

1. 服务器硬件瓶颈：如 CPU、内存、磁盘 I/O 等。
2. 中间件瓶颈：包括参数配置、数据库、Web 服务器等。
3. 应用瓶颈：如 SQL 语句、数据库设计、业务逻辑、算法等。
4. 服务器操作系统瓶颈：如参数配置。
5. 网络瓶颈：对于局域网，通常可以不考虑。

多样性性能测试

性能瓶颈原因

- ## 性能瓶颈点/类别

- [illegible]

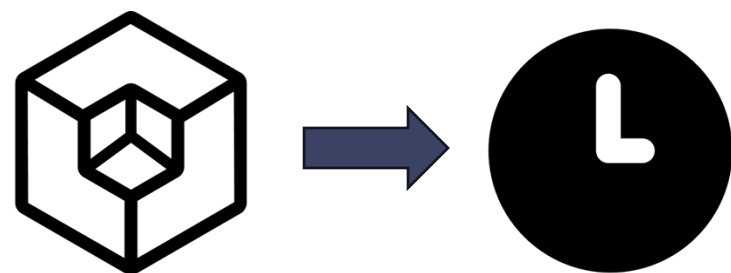
火焰图定位性能瓶颈

性能测试目标与调优

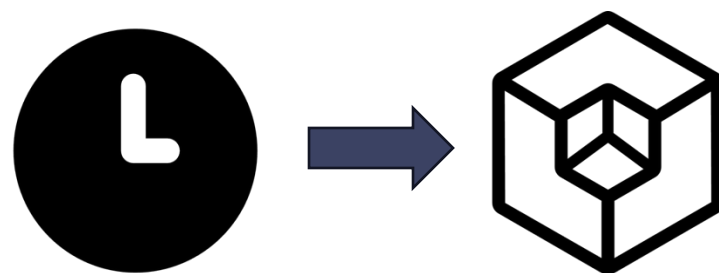
性能测试目标

- 发现性能瓶颈，并提出有效的性能调优方案，以提高被测应用的性能。

常见性能调优策略



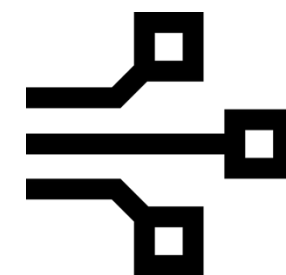
空间换时间



时间换空间



简化代码



并行处理

Web 应用程序与移动端应用程序

Web 应用程序性能测试介绍

Web 应用程序性能测试评估其在速度、服务器响应、网络延迟和数据库查询等方面的表现。通过模拟真实负载，识别性能瓶颈并优化代码和架构，确保应用支持预期负载和用户体验。测试团队与开发人员协作，部署后持续监控性能，重复测试以检测速度、响应和稳定性问题，及时修复缺陷，保证应用高效稳定运行。

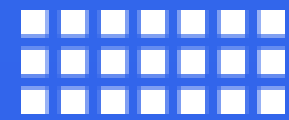
移动端应用性能测试介绍

移动应用测试需模拟真实用户场景，覆盖多样设备、网络和操作系统，确保一致性能。设备硬件、屏幕尺寸和分辨率差异增加了测试复杂性。应用需适配多种屏幕，保持加载一致性和视觉质量。真实设备测试成本高，可通过设定最低硬件要求优化测试范围。

移动端应用特性

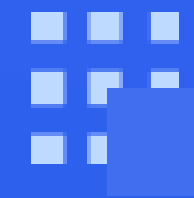
移动端应用测试的特性及其面临的挑战

- 设备与场景多样性：移动设备类型繁杂，应用场景多样，增加了测试的复杂性。
- 性能指标：高 CPU 使用率和电池耗尽可能影响用户体验，甚至导致应用卸载。
- 测试优化：根据目标受众和设备范围，测试人员需选择合适的工具并优化测试流程。例如，仅针对 Android 设备的应用无需在其他操作系统上测试。
- 关键测试点：包括应用启动时间、CPU 和电池使用情况、后台运行等性能指标。
- 跨平台测试：本机应用和基于浏览器的应用需独立测试，后者还需考虑不同浏览器和网络条件。
- 网络条件：移动应用需在不同网络条件下测试，包括 3G、4G、5G 以及不稳定的连接或离线状态，以验证延迟和响应时间是否可接受。



02

故障假设性能测试



故障假设性能测试概述

故障假设性能测试的重要性与作用

- 预见潜在问题：故障假设帮助测试人员预见和识别系统中的潜在问题，从而制定有针对性的测试计划。
- 提出依据：故障假设基于对系统架构、历史故障数据和运行环境的深刻理解，使测试更加聚焦。
- 优化资源分配：由于资源有限，故障假设帮助测试人员确定测试重点，集中精力测试最可能出问题的部分，提高测试效率和效果。
- 提升系统可靠性：通过系统性地提出和验证故障假设，测试人员可以覆盖更多潜在问题，发现隐藏故障，提升系统的可靠性和稳定性。
- 改进系统设计：通过分析和验证故障假设，测试人员能够发现系统设计中的薄弱环节，并提出改进建议。

故障假设是性能测试中不可或缺的工具，帮助测试人员更高效、全面地发现和解决问题，提升系统质量。

性能变异测试

什么是性能变异测试？

性能变异测试是通过引入一组变异算子（对源代码进行修改的操作）来评估和提高性能测试的有效性。每个变异算子应导致程序性能明显下降，同时保留其原有功能。

性能变异测试的目标？

创建与程序正确版本接近但包含性能缺陷的错误版本（变异体），并符合变异测试的两个经典假设：

- 变异体假设：变异体应包含可检测的性能缺陷。
- 测试有效性假设：性能测试应能够有效检测这些变异体中的性能缺陷。

性能测试的角度定义与变异测试相关的经典术语

- **性能变异：**通过语法更改降低程序性能的原始程序变体。保留原始语义的为有效性能变异体，改变功能的为无效性能变异体。功能测试集未检测到的变异体称为潜在有效的，需手动检查确认。
- **杀死性能变异体：**测试检测到显著性能下降时，变异体被“杀死”。使用不同阈值决定何时杀死。
- **活跃变异体：**性能测试未检测到的变异体。需设计可行测试输入（现实且能揭示性能错误）来发现。
- **等价性能变异体：**无测试输入能检测到显著性能差异的变异体。取决于阈值和测试输入的可行性。
- **性能变异评分：**评估测试错误揭示能力的指标，计算为被杀死的变异体与非等价变异体的比率。提高评分可提升测试集质量。

通用语言功能和非并发程序（变异可能导致同步问题） 示例

定义了七个性能变异算子（如表8.3所示），影响编程语言中的基本功能（如循环、条件语句、容器、对象等）。这些算子可以适应不同语言的语法。

表 8.3: 性能变异算子

算子	类型	性能因素
RCL	循环扰动	执行时间
URV	方法调用	执行时间
MSL	对象生成	内存消耗
	条件修改	执行时间
	循环扰动	执行时间
SOC	条件修改	执行时间
HWO	方法调用	执行时间
CSO	对象生成	内存消耗
MSR	集合变换	内存消耗

通用语言功能和非并发程序（变异可能导致同步问题） 示例

表8.4给出了一个 RCL 示例，这里的前提条件是当存在其他可以在某个时刻停止循环的条件时，可以生成 RCL 性能变异。当循环的控制表达式不为空（情况 1和 2 或循环的控制表达式中有多个条件（情况 3）时，就会发生这种情况。

表 8.4: RCL 性能变异示例

```
1 原始代码:
2  bool b = false;
3  while(i<n && !b ){
4      .....
5      if (l[i]==1)
6          b=true;
7      .....
8  }
```

```
1 变异代码:
2  bool b = false;
3  while(i<n){
4      .....
5      if (l[i]==1)
6          b=true;
7      .....
8  }
```

通用语言功能和非并发程序（变异可能导致同步问题） 示例

如表8.5所示，min 变量的删除以及通过调用 minValue 函数替换对该变量的引用，强制重新计算 $v12 \times n$ 次。这里的前提条件是当（1）存在一个存储计算值的变量并且它被多次引用（等价的情况），以及（2）存储计算值的变量和计算中涉及的变量在计算点和变量引用之间不会发生变化。

表 8.5: URV 性能变异示例

```
1 原始代码:
2  int min=minValue(v1);
3  for(int i =0;i<n;i++){
4      if (v2[i]>min)
5          v2[i]=min;
6  }
```

```
1 变异代码:
2  for(int i =0;i<n;i++){
3      if (v2[i]>minValue(v1))
4          v2[i]=minValue(v1);
5  }
```


通用语言功能和非并发程序（变异可能导致同步问题） 示例

在循环中生成许多临时对象只是为了提供简单服务（例如，为其他对象的字段赋值）的情况。从未使用和很少使用的分配（即创建但几乎不被引用的对象）出现的频率可能很高。基于上述性能缺陷，MSL 算子（表 8.6）会搜索循环语句之前生成新对象的代码部分。

表 8.6: MSL 性能变异示例

1 原始代码:

2 Foo ol;

3 for (int i=0;i<n;i++){

4 v[i]=v[i]+ol.getField();

5 }

1 变异代码:

2 for (int i=0;i<n;i++){

3 Foo ol;

4 v[i]=v[i]+ol.getField();

5 }

通用语言功能和非并发程序（变异可能导致同步问题）示例

在有多个条件的情况下，需要更多时间评估的条件通常放在最后一个位置。这可以避免根据第一个较轻条件的结果来评估最耗时的条件。可跳过 RCL 算子并 COR 算子，SOC 算子强制评估可能不需要的条件，具体取决于其他条件的结果。因此，SOC算子（表8.7）会交换由二元逻辑算子（&& 和 ||）链接的条件中的操作数，以便在不考虑其他条件的情况下评估最耗时的条件。

表 8.7: SOC 性能变异示例

```
1 原始代码:  
2  if (a!=1||costlyMethod()){  
3      .....  
4  }
```

```
1 变异代码:  
2  if (costlyMethod()||a!=1){  
3      .....  
4  }
```

通用语言功能和非并发程序（变异可能导致同步问题） 示例

HWO 算子（表8.8）在每次调用第三方库中的方法和已知的重量级操作（存储访问、网络连接...）后立即注入延迟 t ，以便揭示工作负载问题。

表 8.8: HWO 性能变异示例

原始代码:

`costlyMethod(a1, a2);`

.....

`}`

变异代码:

`costlyMethod(a1, a2);``sleep_for(std::chrono::seconds(t));`

.....

`}`

通用语言功能和非并发程序（变异可能导致同步问题） 示例

在某些情况下，可以在多次调用同一方法时重用单个对象。CSO 算子（表8.9）在每次调用方法时强制生成一个新对象，而不是重用已创建的对象。

表 8.9: CSO 性能变异示例

1 原始代码:

```
2 Foo multiply(Foo f, int n ){  
3     return f.getNumber() * n;  
4 }
```

1 变异代码:

```
2 Foo multiply(Foo f, int n ){  
3     Foo f2(f);  
4     return f2.getNumber() * n;  
5 }
```

通用语言功能和非并发程序（变异可能导致同步问题） 示例

基于从未使用和很少使用的分配错误模式以及集合利用率低的原因，MSR 算子（表8.10）为动态集合分配更少或更多的空间。该算子通过动态分配来修改集合，以缩小或扩大元素的保留空间，以模拟这两种情况。

表 8.10: MSR 性能变异示例

```
1 原始代码:  
2  vector<Foo>v(INIT_SIZE);
```

```
1 变异代码:  
2  vector<Foo>v( 1 );  
3  -----  
4  变异代码:  
5  vector<Foo>v( INIT_SIZE * t );
```